

◆ CS EDUCATION ◆



쿠키, 세션, 토큰

25.12.26 교육팀 김민아

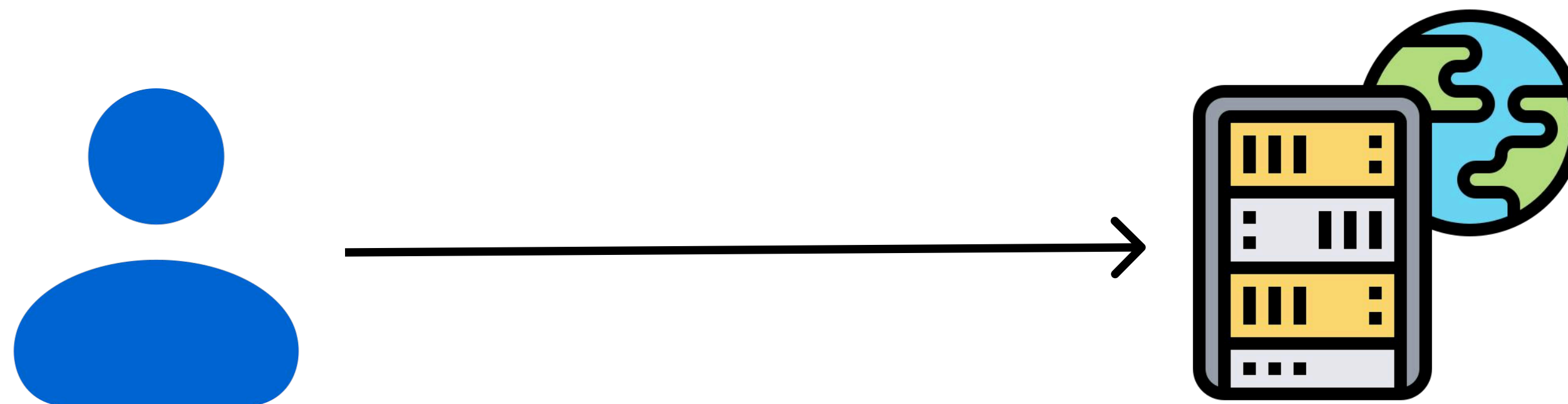
교육 목표

1. 쿠키, 세션, 토큰에 대한 이해
2. JWT에 대한 이해

인증/인가 시스템은 왜 필요한가요?

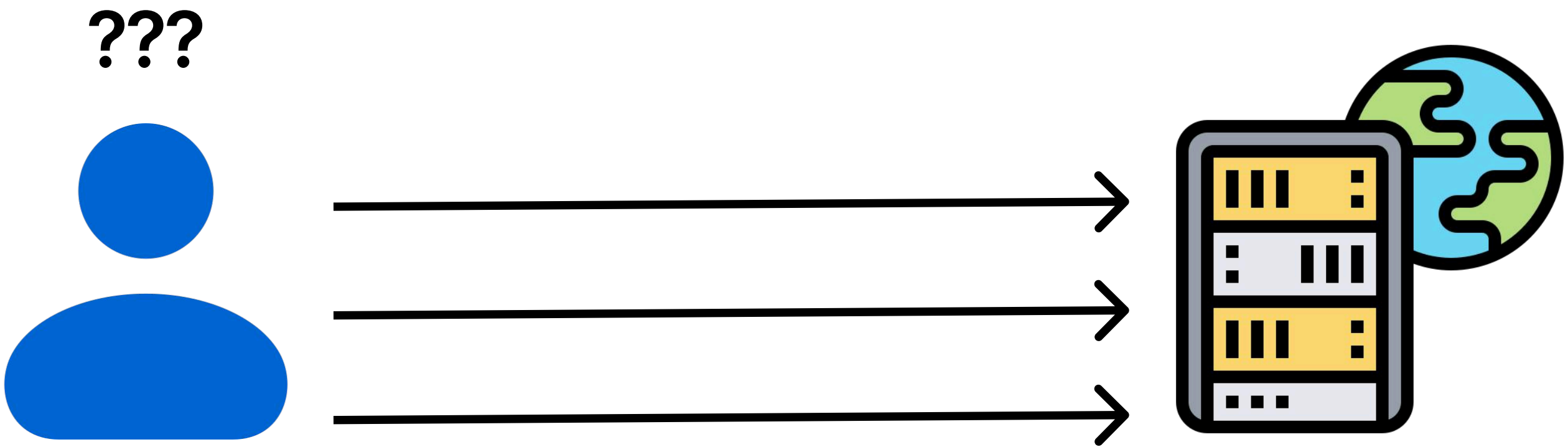
HTTP는 **무상태 (stateless)**

서버는 이전 요청을 기억하지 못함



상태를 기억하지 못하면?

- 로그인 유지 불가
- 사용자별 데이터 구분 불가
- 권한(사용자/관리자) 판단 불가



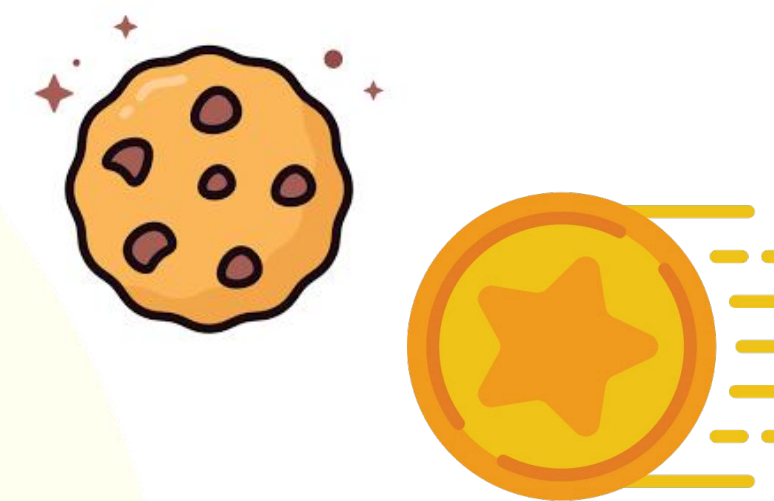
인증/인가 시스템

사용자를 식별 (Authentication)

권한을 검증 (Authorization)

요청 간 상태를 유지

쿠키, 세션, 토큰



HTTP의 stateless 특성을 보완하고 상태를 유지하기 위한 수단

쿠키란?

- 웹사이트가 사용자의 브라우저에 저장하는 작은 데이터
 - key-value 형태의 텍스트
 - 서버 또는 클라이언트(JS)에서 생성 가능

쿠키란?



DevTools - www.google.com/

Elements Console Sources Lighthouse Network Performance **Application** >> ⚙️ ⋮

Storage

- ▶ Local Storage
- ▶ Session Storage
- IndexedDB
- Web SQL
- ▼ Cookies

Filter ☐ Only show cookies with an issue

Name	Value	D..	P...	E...	S...	H..	S...	S...	P▼	P...	S...	S...
CONSENT	PENDING+188	...	/	2...	18		✓			M...	S...	4...
__Secure-...	10.SE=nt91XCBmj...	...	/	2...	1...	✓	✓	L...		M...	S...	4...
AEC	ARSKqsLi1w3e5SN...	...	/	2...	61	✓	✓	L...		M...	S...	4...

▶ **https://www.google.com**

- Trust Tokens
- Interest Groups
- ▼ Shared Storage
 - https://www.google.com
- Cache Storage

Cookie Value ☐ Show URL decoded

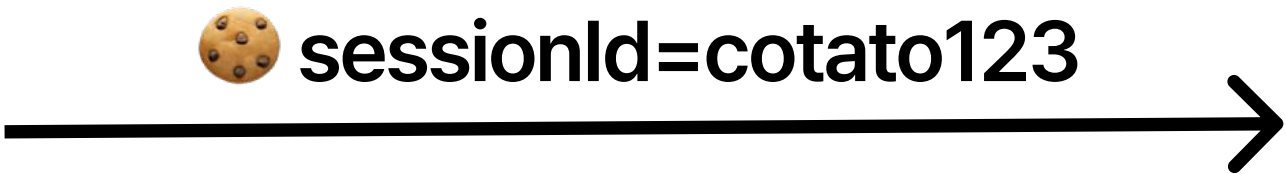
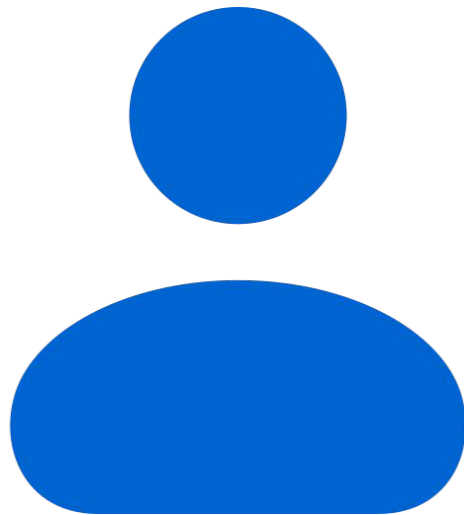
PENDING+188

쿠키의 목적

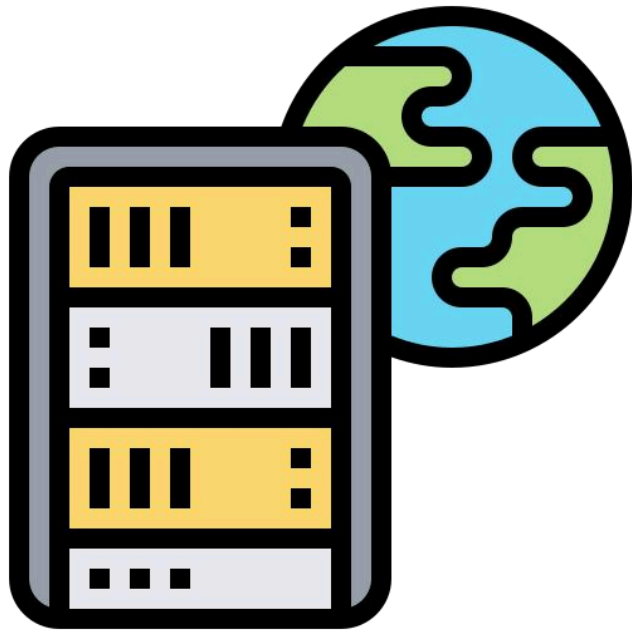
- 1. 사용자 세션 유지
- 2. 개인화
- 3. 추적

쿠키의 목적 (1) 사용자 세션 유지

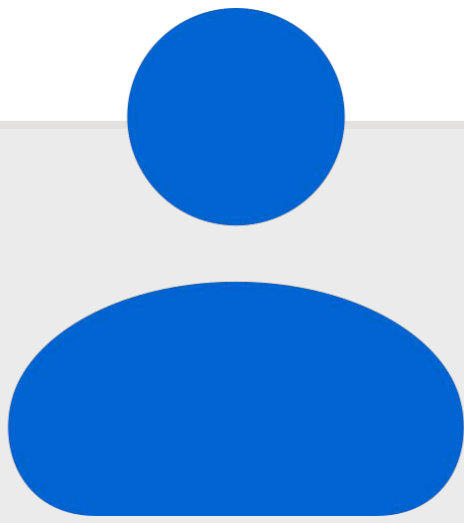
클라이언트: 요청마다 쿠키 전송



서버: 쿠키로 사용자 식별



쿠키의 목적 (2) 개인화



언어 설정
다크모드
사용자 환경 유지

쿠키의 목적 (3) 사용자 추적

- 방문 페이지
- 체류 시간
- 클릭 패턴



광고 맞춤화
서비스 개선

쿠키 인증 방식

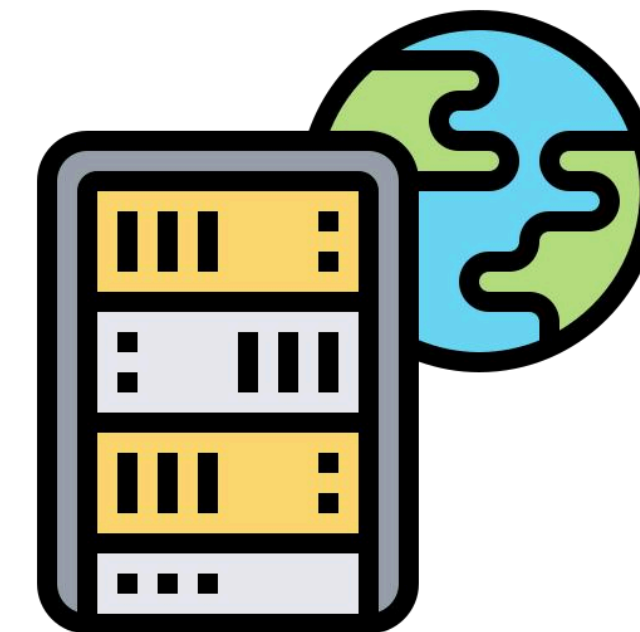
클라이언트가 사용자 인증 정보를 관리!



로그인 요청

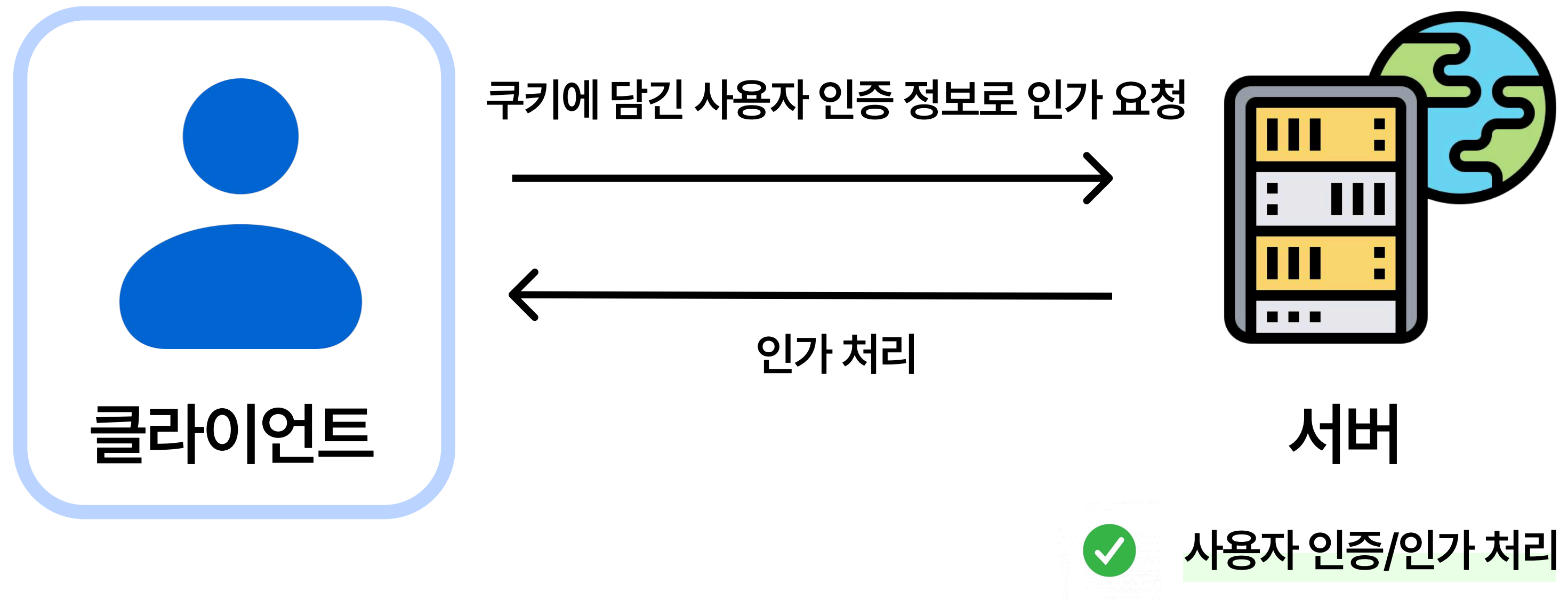


쿠키에 사용자 인증 정보 담아서 응답



쿠키 저장

쿠키 인증 방식



쿠키 인증 방식의 문제점

- (1) 클라이언트가 쿠키에 담긴 정보를 위조할 수 있음
→ CSRF, XSS 공격에 취약
- (2) 쿠키의 데이터 크기가 제한적

세션이란?

- 사용자 상태를 서버에 저장 (Stateful)
- 클라이언트에는 세션 ID만 쿠키로 저장
- 세션은 쿠키를 기반으로 인증하는 방식

세션 인증 방식



✓ sessionID 저장

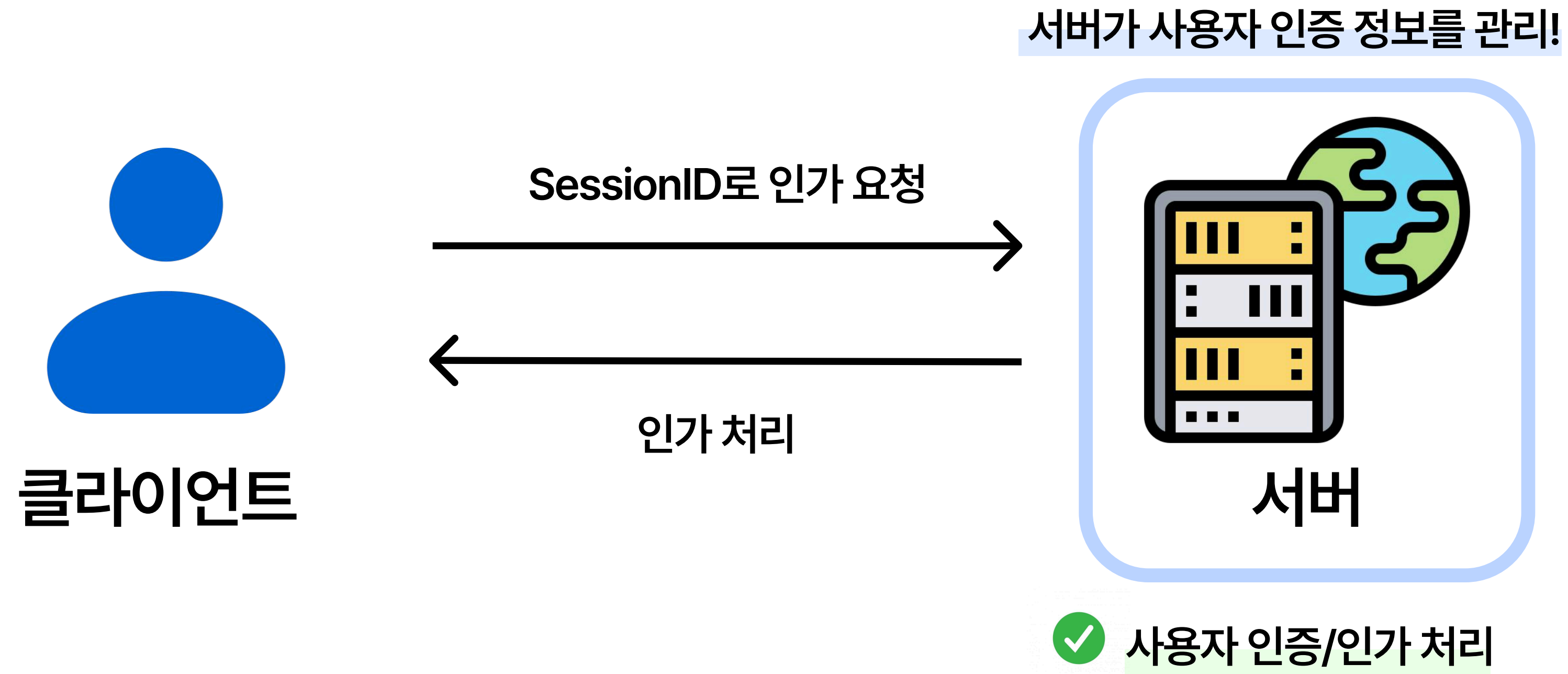


서버가 사용자 인증 정보를 관리!



✓ 사용자 인증 정보 저장

세션 인증 방식



세션 인증 방식의 문제점

- (1) 서버가 사용자 인증 정보를 관리하기 때문에 메모리 부하 증가
- (2) 쿠키를 포함한 요청이 외부에 노출될 수 있는 가능성



토큰이란?

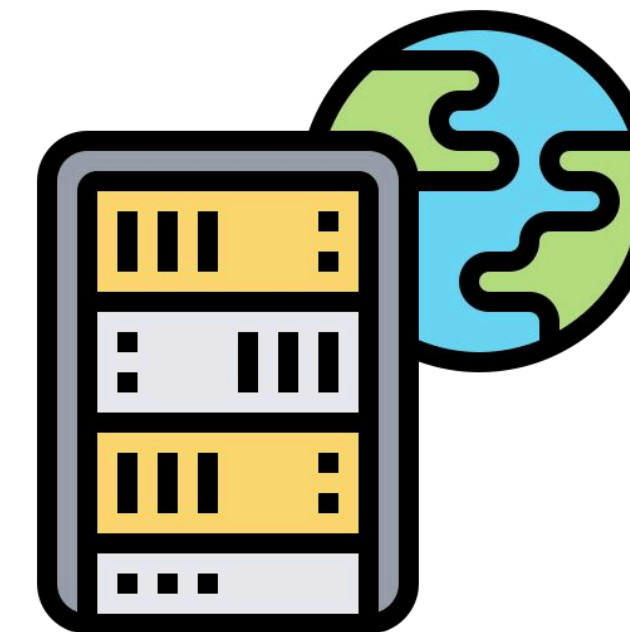
- 사용자를 증명하는 문자열
- 서버는 상태를 저장하지 않음
- 클라이언트가 토큰을 들고 다님 (stateless)

토큰 인증 방식

클라이언트가 사용자 인증 정보를 관리!



✓ token 저장



서버

토큰 인증 방식

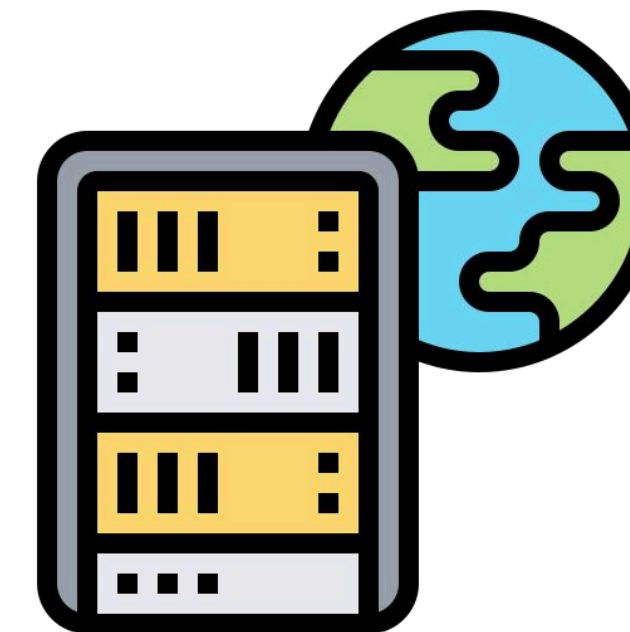
클라이언트가 사용자 인증 정보를 관리!



token으로 인가 요청



인가 처리



서버



token 검증

사용자 인증/인가 처리

JWT(JSON Web Token)

- JSON 형식의 데이터를 안전하게 주고받기 위한 개방형 표준
- 사용자의 인증 정보와 서버의 Secret Key로 서버가 생성한 토큰
- 헤더, 페이로드, 서명의 세 부분으로 구성

XXXXXX.YYYYYY.ZZZZZZ

헤더(Header) 내용(Payload) 서명(Signature)

헤더(Header)	내용(Payload)	서명(Signature)
<pre>{ "alg": "HS256", "typ": "JWT" }</pre>	<pre>{ "sub": "1234546", "name": "kim", "iat": 1442 }</pre>	<pre>HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), your-256-bit-secret)</pre>

XXXXXX.YYYYYY.ZZZZZZ

헤더(Header) 내용(Payload) 서명(Signature)

헤더(Header)	내용(Payload)	서명(Signature)
토큰의 유형 서명 알고리즘	사용자 정보 사용자 ID 권한 만료 시간	헤더의 서명 알고리즘과 비밀 키를 사용하여 생성

JWT를 이용한 인증 방식

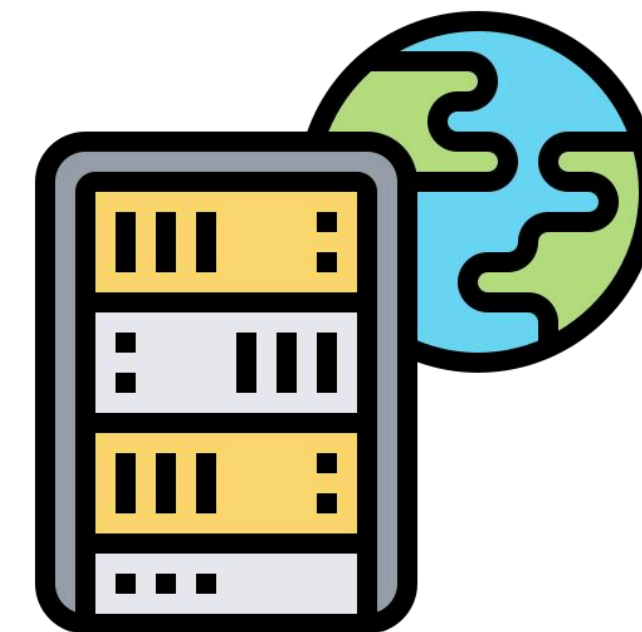
클라이언트가 사용자 인증 정보를 관리!



로그인 요청



JWT(access token, refresh token) 발급



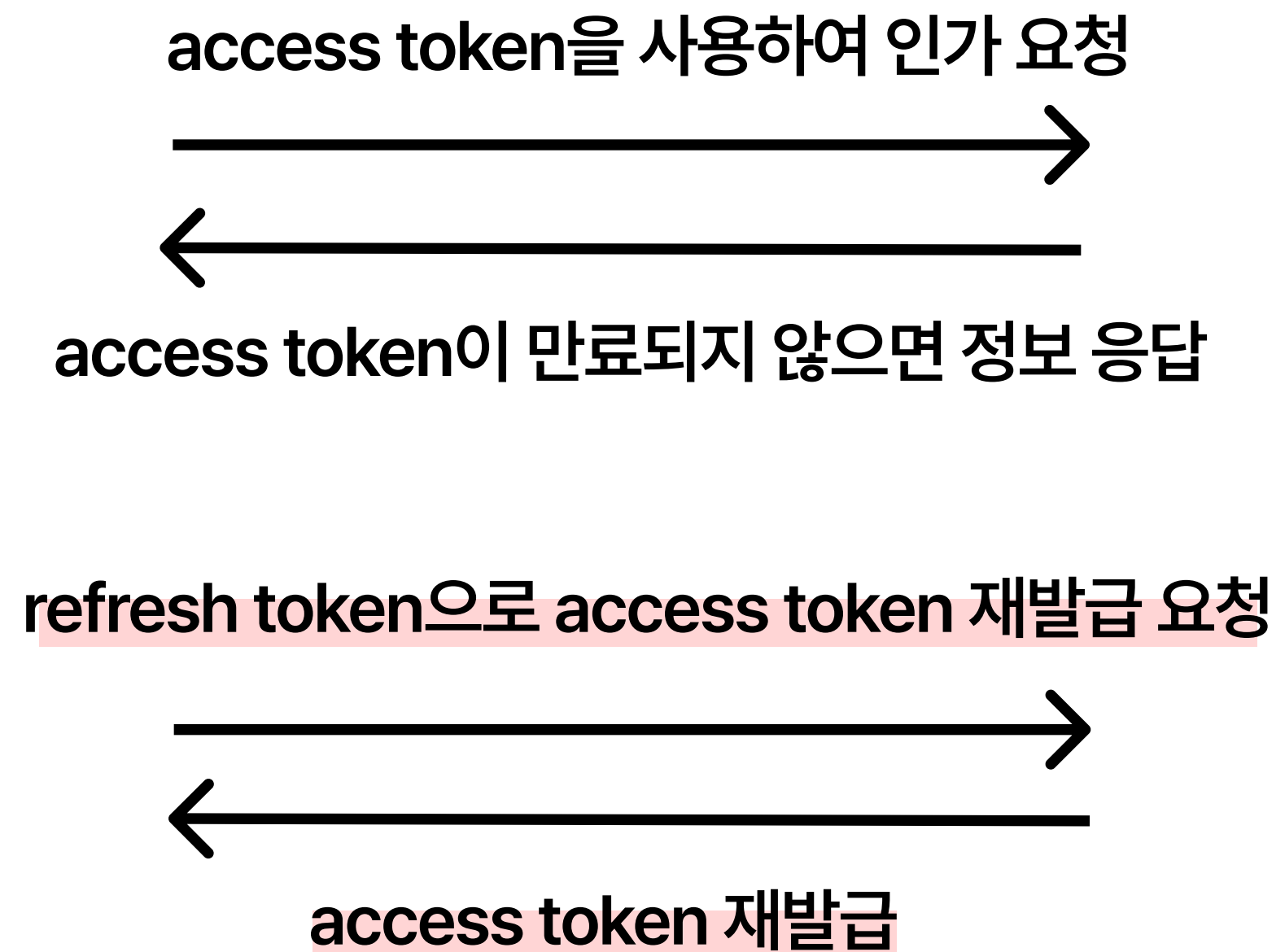
서버



access token, refresh token 저장

JWT를 이용한 인증 방식

클라이언트가 사용자 인증 정보를 관리!



?

**프로젝트에 어떤 인증 방식을 사용할까요?
클라이언트는 토큰을 어디에 저장해야 할까요?**